pfi

Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

pandas: Series and DataFrames, Part 2

WEEK 2, DAY 2, PART 2

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

The desk lead's eleventh note

The file is loaded; now make it answer. What is in the book, sector by sector? Which bonds pass a test: Energy, below par, long and cheap? The widest spreads, the biggest positions. Mark BEZL 99039NAC0 at 97.00 on a copy, not our record, and tell me what it costs. Then send the widest spreads in a table I can forward.

```
holdings = pd.read_csv(data_path)
by_cusip = holdings.set_index('cusip')
print(holdings.shape, desk.shape)
(200, 19) (4, 6)
```

A new notebook starts empty. The issuers are invented.

Same 200 bonds, same desk: every answer checks against Part 1.

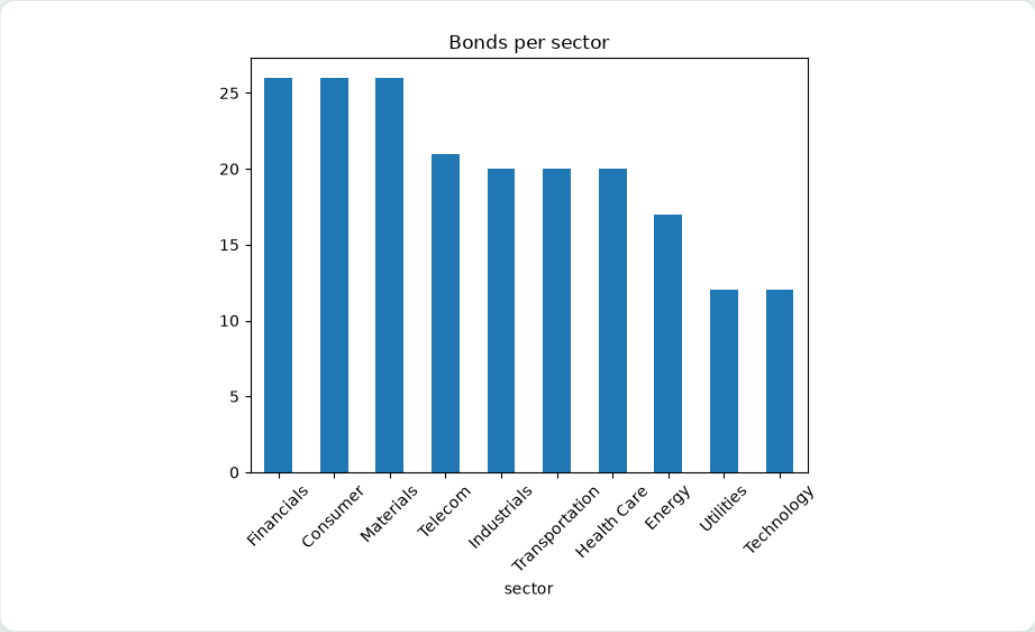
7.6 Totals, the middle, and bonds per sector

```
print('Market value:', holdings['market_value'].sum())
print('Face held:', holdings['par_held'].sum())
print('OAS mean:', holdings['oas'].mean(), 'median:', holdings['oas'].median())
print('Price low:', holdings['price'].min(), 'high:', holdings['price'].max())
```

```
Market value: 490819215.0
Face held: 489000000
OAS mean: 185.75 median: 135.5
Price low: 62.045 high: 119.131
```

```
sector_counts = holdings['sector'].value_counts()
sector_counts
```

```
sector
Financials        26
Consumer          26
Materials         26
Telecom           21
Industrials       20
Transportation    20
Health Care       20
Energy            17
Utilities         12
Technology        12
Name: count, dtype: int64
```




```
chart = sector_counts.plot.bar(rot=45, title='Bonds per sector')
```

Quote the median beside the mean, as 7.4's spreads showed.

Shares, the most common, each value once

```
print(holdings['sector'].unique().tolist())
print(holdings['as_of_date'].unique().tolist())
print('Tickers:', holdings['ticker'].nunique(), 'CUSIPs:', holdings['cusip'].nunique())
['Financials', 'Industrials', 'Transportation', 'Utilities', 'Consumer', 'Materials', 'Technology', 'Telecom', 'Health Care', 'Energy']
['2026-09-30']
Tickers: 115 CUSIPs: 200
```

```
holdings['sector'].value_counts(normalize=True)
sector
Financials        0.130
Consumer          0.130
Materials         0.130
```

```
print('Most common rating:', holdings['rating'].mode().iloc[0])
Most common rating: A-
```

With no tie, mode() is the top row of value_counts().

Excel side by side

pandas	Excel
`holdings['market_value'].sum()`	=SUM(S2:S201)
`holdings['oas'].mean()`	=AVERAGE(M2:M201)
`holdings['oas'].median()`	=MEDIAN(M2:M201)
`holdings['price'].min()`	=MIN(I2:I201)
`holdings['sector'].value_counts()`	a pivot table, sector in Rows, a count in Values
`value_counts(normalize=True)`	the same, Show Values As, % of Grand Total
`holdings['sector'].unique()`	=UNIQUE(E2:E201) (Excel 2021, Microsoft 365)
`holdings['ticker'].nunique()`	=COUNTA(UNIQUE(C2:C201)) (Excel 2021, Microsoft 365)

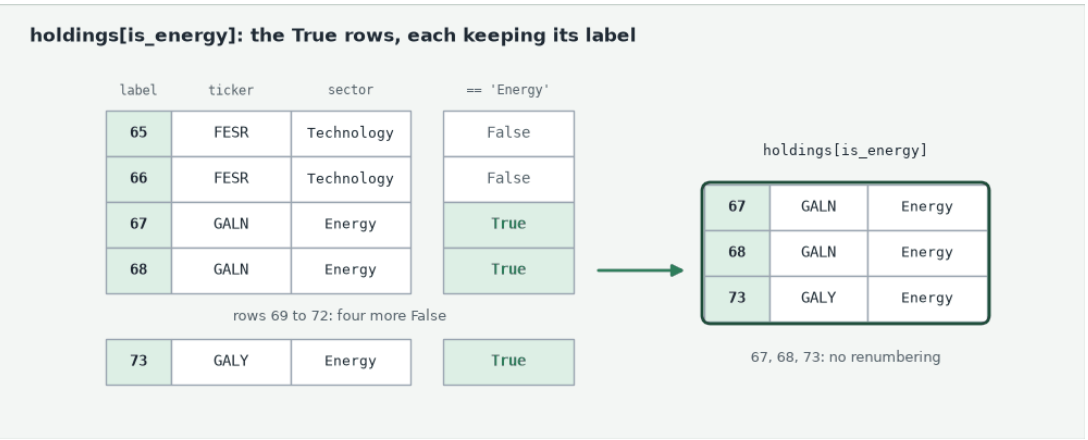
One line per figure, naming the column, not a range. Now pick the rows.

7.7 A test on a column picks rows

```
is_energy = holdings['sector'] == 'Energy'
is_energy.iloc[65:69]
65    False
66    False
67     True
68     True
Name: sector, dtype: bool
```

```
energy = holdings[is_energy]
energy[['ticker', 'issuer', 'rating', 'price', 'oas']].head()
   ticker               issuer rating    price  oas
67   GALN  Galvostrin Resources   BBB+   86.621   99
68   GALN  Galvostrin Resources   BBB+   89.089   88
73   GALY   Galvolvey Resources     A+   93.393   74
82   HESR   Heskentor Resources   BBB-  112.884  155
86   HULR     Hulvimer Pipeline    BBB   93.030  110
```

```
print('Energy bonds:', is_energy.sum())
print('Energy market value:', holdings.loc[is_energy, 'market_value'].sum())
Energy bonds: 17
Energy market value: 38830005.0
```



A test on a column picks rows, as NumPy's masks picked bonds (6.5).

A filter keeps each row's label.

Below par, both tests, or either

```
long_and_low = (holdings['duration'] > 10) & (holdings['price'] < 90)
print('Count:', long_and_low.sum())
holdings.loc[long_and_low, ['ticker', 'coupon', 'maturity', 'price', 'duration']]
Count: 6
    ticker  coupon    maturity   price  duration
36    DNMR   5.250  2047-06-05  89.056    11.764
84    HLVF   7.625  2053-06-08  88.366    10.218
105   MORX   6.125  2055-12-21  85.722    12.139
117   NEVX   3.875  2048-01-06  73.873    12.898
150   SYLX   3.625  2055-09-03  76.903    16.042
184   WYXA   6.750  2056-02-23  86.884    11.558
```

```
low_or_wide = (holdings['price'] < 80) | (holdings['oas'] > 1000)
print('Count:', low_or_wide.sum())
holdings.loc[low_or_wide, ['ticker', 'rating', 'price', 'oas']]
Count: 4
    ticker rating   price   oas
17    CLDB      B  62.045  1446
65    FESR   CCC+  87.116  1551
117   NEVX   BBB-  73.873   153
150   SYLX     A+  76.903    59
```



The word "and" fails here as in NumPy 6.5: "The truth value of a Series is ambiguous".

```
print('Below par:', (holdings['price'] < 100).sum())
Below par: 113
```

& narrows, | widens, and a swap raises no error: check the count.

One of several ratings, and the filters in Excel

```
single_a = holdings['rating'].isin(['A+', 'A', 'A-'])
print('Single-A bonds:', single_a.sum())
print('Average OAS:', round(holdings.loc[single_a, 'oas'].mean(), 2))
Single-A bonds: 56
Average OAS: 79.41
```

79.41 bp against 185.75 for the whole book.

Excel side by side. Sector in E, rating in F, price in I, OAS in M, duration in N, market value in S.

Question	pandas	Excel
Show the Energy bonds	`holdings[is_energy]`	AutoFilter on the sector column
Count them	`is_energy.sum()`	=COUNTIF(E:E, "Energy")
Their market value	`holdings.loc[is_energy, 'market_value'].sum()`	=SUMIF(E:E, "Energy", S:S)
Average OAS of one rating	`holdings.loc[holdings['rating'] == 'A', 'oas'].mean()`	=AVERAGEIF(F:F, "A", M:M)
Both tests	`long_and_low.sum()`	=COUNTIFS(N:N, ">10", I:I, "<90")
One of several ratings	`single_a.sum()`	=SUM(COUNTIF(F:F, {"A+","A","A-"}))

The filter is in writing: read, reviewed, rerun. Now put the book in order.

7.8 The widest spreads, the largest positions

```
widest = holdings.sort_values('oas', ascending=False)
widest[['ticker', 'sector', 'rating', 'price', 'oas']].head()
```

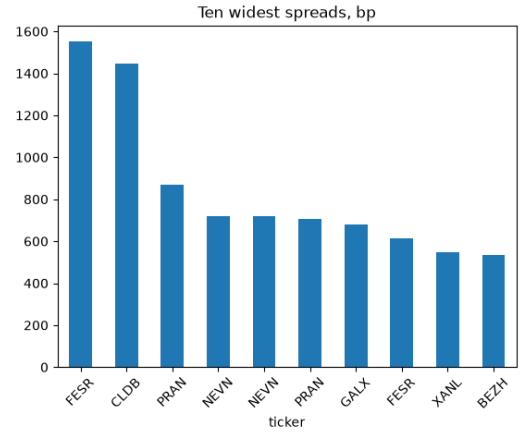
	ticker	sector	rating	price	oas
65	FESR	Technology	CCC+	87.116	1551
17	CLDB	Consumer	B	62.045	1446
132	PRAN	Financials	CCC+	89.407	871
110	NEVN	Utilities	B-	96.616	720
111	NEVN	Utilities	B-	94.053	719

```
holdings.nlargest(5, 'market_value')[['ticker', 'par_held', 'price', 'market_value']]
```

	ticker	par_held	price	market_value
0	BEZE	5000000	106.727	5435500.0
59	FESA	5000000	106.471	5422700.0
103	LORN	4750000	109.248	5287462.5
194	ZARL	5000000	101.481	5253950.0
104	LORX	5000000	104.801	5241650.0

```
holdings.nsmallest(3, 'price')[['ticker', 'coupon', 'maturity', 'price']]
```

	ticker	coupon	maturity	price
17	CLDB	9.500	2034-12-06	62.045
117	NEVX	3.875	2048-01-06	73.873
150	SYLX	3.625	2055-09-03	76.903



Excel side by side

Data, Sort reorders the sheet itself; =SORT(A2:S201, 13, -1) in Excel 2021 and Microsoft 365 spills a sorted copy, as sort_values() hands one back. =LARGE(S2:S201, 1) and =SMALL(I2:I201, 1) give one value each; nlargest() and nsmallest() give whole rows.

FESR, PRAN, and NEVN each appear twice: a ticker is not a bond.

7.9 Coupon income, weights, a column from a list

```
holdings['coupon_income'] = holdings['par_held'] * holdings['coupon'] / 100
print('Total annual coupon income:', holdings['coupon_income'].sum())
holdings[['ticker', 'coupon', 'par_held', 'coupon_income']].head(3)
```

```
Total annual coupon income: 28864375.0
  ticker  coupon  par_held  coupon_income
0   BEZE   7.000   5000000       350000.0
1   BEZH   8.125    250000        20312.5
2   BEZL   5.125   1500000        76875.0
```

```
desk_scores = desk.copy()
desk_scores['Score'] = [9, 11, 7, 14]
print(desk_scores[['Ticker', 'Rating', 'Score']])
```

```
  Ticker Rating  Score
0   QVNE    BBB      9
1   DNMR    BB+     11
2   PLWK     A-      7
3   HLVF     B+     14
```

```
holdings['weight_pct'] = holdings['market_value'] / holdings['market_value'].sum() * 100
print('Sum of the weights:', round(holdings['weight_pct'].sum(), 6))
holdings.nlargest(3, 'weight_pct')[['ticker', 'market_value', 'weight_pct']]
```

```
Sum of the weights: 100.0
    ticker  market_value  weight_pct
0     BEZE     5435500.0    1.107434
59    FESA     5422700.0    1.104826
103   LORN     5287462.5    1.077273
```

Weights adding up to 100 is the check on any weight column. BEZE is about 1.11 percent, against 0.5 for an equal split.

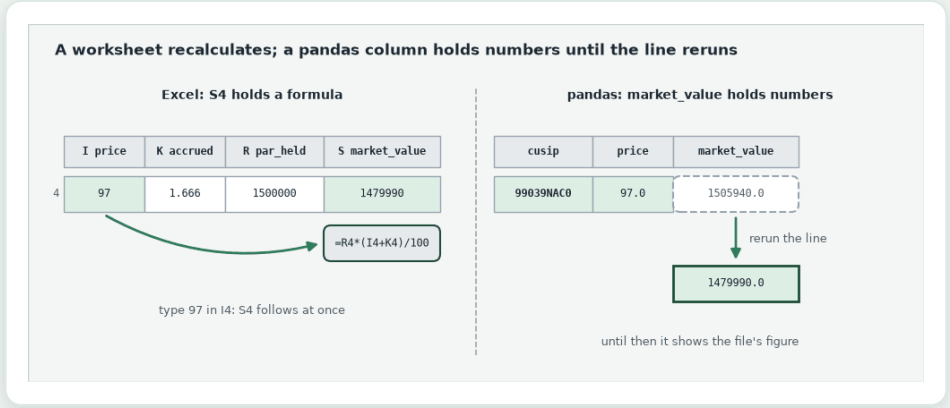
Add each position's annual coupon income: face held times coupon, divided by 100.

The note's mark, on a copy

```
what_if = by_cusip.copy()
what_if.loc['99039NAC0', 'price'] = 97.0
print('What-if:', what_if.loc['99039NAC0', 'price'])
print('Record:', by_cusip.loc['99039NAC0', 'price'])
What-if: 97.0
Record: 98.73
```

```
print(what_if.loc['99039NAC0', 'market_value'])
what_if['market_value'] = what_if['par_held'] * (what_if['price'] + what_if['accrued']) / 100
change = what_if['market_value'].sum() - by_cusip['market_value'].sum()
print('Change in market value:', round(change, 2))
1505940.0
Change in market value: -25950.0
```

Without .copy(), what_if = by_cusip gives one table two names (Day 3), and 97 lands on the record.



1,500,000 of face marked 1.73 points lower: 1,500,000 times 1.73, divided by 100.

A column holds values, not formulas: change an input, rerun the line.

Every row that passes a test, and filter then copy

```
marks = holdings.copy()
marks['price_check'] = 'OK'
marks.loc[marks['price'] < 80, 'price_check'] = 'Confirm'
marks.loc[marks['price'] < 80, ['ticker', 'price', 'price_check']]
    ticker   price price_check
17    CLDB  62.045     Confirm
117   NEVX  73.873     Confirm
150   SYLX  76.903     Confirm
```

With .copy() every version agrees: filter, then copy, before changing anything.

```
energy_what_if = holdings[is_energy].copy()
energy_what_if['price'] = energy_what_if['price'] - 2
print(f"What-if average price: {energy_what_if['price'].mean():.3f}")
print(f"Record average price:  {holdings.loc[is_energy, 'price'].mean():.3f}")
What-if average price: 95.407
Record average price:  97.407
```

```
positional = holdings.copy()
positional.iloc[0, 8] = 100.0
print('Copy:', positional.loc[0, 'price'], 'Record:', holdings.loc[0, 'price'])
Copy: 100.0 Record: 106.727
```

Excel side by side

Step	pandas	Excel
Add a column	`holdings['par_held'] * holdings['coupon'] / 100`	=R2*G2/100, filled down
Weight	`holdings['market_value'] / holdings['market_value'].sum() * 100`	=S2/SUM(S:S)*100, filled down
Work on a copy	`by_cusip.copy()`	Move or Copy, Create a copy
Overwrite one value	`what_if.loc['99039NAC0', 'price'] = 97.0`	type over the cell
Set the rows that pass a test	`marks.loc[marks['price'] < 80, 'price_check'] = 'Confirm'`	=IF(I2<80, "Confirm", "OK"), filled down
Copy filtered rows	`holdings[is_energy].copy()`	copy what an AutoFilter shows: only those rows are copied
Rename, remove	`rename(), drop()`	retype the header; Delete

Predict first: decide, then run each in the notebook

Predict 1

The desk sorts the holdings by rating. Where does AAA land? A) First. B) Last. C) Somewhere in the middle.

```
by_rating = holdings.sort_values('rating')
print(by_rating['rating'].unique().tolist())
```

Predict 2

The next cell hands the same table three scores for its four bonds. What happens? A) The fourth row gets NaN. B) The list starts again, so the fourth row gets 9. C) An error.

```
desk_scores['Score'] = [9, 11, 7]
```

Predict 3

"what_if" is a copy of "by_cusip", made from "holdings" in the first cell; "holdings" has gained two columns since. How many columns does "what_if" have before and after the two date columns go? A) 21, then 19. B) 18, then 16. C) 20, then 18.

```
print('Before:', what_if.shape)
what_if = what_if.drop(columns=['as_of_date', 'price_date'])
print('After:', what_if.shape)
```

Predict 4

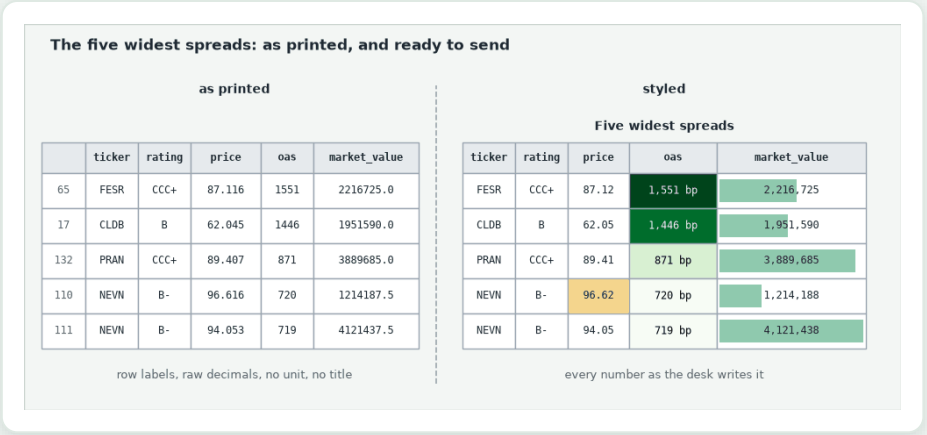
Older tutorials also write "inplace=True". What does "result" hold after the next cell? A) A new table without the Face column. B) None.

```
result = older.drop(columns=['Face'], inplace=True)
print('Returned:', result)
print('Columns now:', older.columns.tolist())
```

7.10 A table worth forwarding

```
widest5 = holdings.nlargest(5, 'oas')[['ticker', 'rating', 'price', 'oas', 'market_value']]
(widest5.style.format({'price': '{:.2f}', 'oas': '{:,.0f} bp', 'market_value': '{:,.0f}'})
    .background_gradient(subset=['oas'], cmap='Greens')
    .highlight_max(subset=['price'], color='#f4d58d')
    .bar(subset=['market_value'], color='#8fc9ae')
    .set_caption('Five widest spreads')
    .hide(axis='index'))
```

.style formats the display, as Excel's Format Cells and Conditional Formatting do. Display only: widest5 keeps every digit, as a cell formatted to two decimals still holds 87.116; round() changes the number itself.



	ticker	rating	price	oas	market_value
65	FESR	CCC+	87.116	1551	2216725.0
17	CLDB	B	62.045	1446	1951590.0
132	PRAN	CCC+	89.407	871	3889685.0
110	NEVN	B-	96.616	720	1214187.5
111	NEVN	B-	94.053	719	4121437.5

ticker	rating	price	oas	market_value
FESR	CCC+	87.12	1,551 bp	2,216,725
CLDB	B	62.05	1,446 bp	1,951,590
PRAN	CCC+	89.41	871 bp	3,889,685
NEVN	B-	96.62	720 bp	1,214,188
NEVN	B-	94.05	719 bp	4,121,438

Excel side by side

pandas	Excel
`.style.format({'price': '{:.2f}'})`	Format Cells, Number, 2 decimal places
`.style.format({'oas': '{:,.0f} bp'})`	Format Cells, Custom, #,##0" bp"
`.background_gradient(cmap='Greens')`	Conditional Formatting, Color Scales
`.highlight_max()`	Conditional Formatting, Top/Bottom Rules, Top 10 Items, set to 1
`.bar()`	Conditional Formatting, Data Bars
`.set_caption('...')`	a title in the row above the table
`.hide(axis='index')`	View, untick Headings: row numbers and column letters both go

The note, answered

The note asked	Answer
Sector by sector	ten sectors, 12 to 26 bonds each
Energy, below par, long and cheap	17 Energy bonds worth 38,830,005; 113 below par; six long and cheap
Widest spreads, biggest positions	FESR at 1,551 bp; BEZE at 5,435,500
BEZL at 97.00 on a copy	25,950 off the market value; the record still at 98.73
A table to forward	the widest spreads, formatted, with a caption

Be proud: one line per question turned two hundred bonds into answers, and no price in the record moved.

Week 2's Day 3 brings separate files: join them, stack them, and total the book by sector, the market value share this notebook could only count.